Innovación en Formación Profesional

Proyecto de big data e inteligencia artificial

Fase 1

CURSO DE ESPECIALIZACIÓN EN

INTELIGENCIA ARTIFICIAL Y BIG DATA

MODALIDAD ONLINE

Daniel Chacón Bautista

Javier Quiñonero González

David Bartol Cortés

Álvaro Benito Diaz

Documentación y diseño técnico.....3

1. Título del proyecto + Integrantes del equipo.....3

2. Resumen ejecutivo.....3

3. Descripción funcional del sistema.....4

4. Alcance definitivo (IN / OUT).....5

5. Arquitectura del sistema (alto nivel).....5

6. Tecnologías seleccionadas.....8

7. Modelo de datos.....9

8. Datos.....11

9. Ética, legalidad y accesibilidad.....13

10. Backlog inicial.....14

11. Plan de trabajo.....17

12. Estrategia de desarrollo.....19

13. Riesgos técnicos actualizados.....20

14. Repositorio del proyecto.....21

Documentación y diseño técnico

1. Título del proyecto + Integrantes del equipo

EmoGestureAI

Integrante	Rol Principal	Responsabilidad clave
Javier Quiñonero	Data Engineer	Captura del dataset y extracción de coordenadas (MediaPipe).
David Bartol	ML Engineer	Entrenamiento de modelo y validación de métricas.
Daniel Chacón	Platform Architect	Integración cámara-IA, lógica de juego e interfaz (OpenCV)
Álvaro Benito	BI & QA Analyst	Registro de partidas, KPIs y dashboard de rendimiento

2. Resumen ejecutivo

EmoGestureAI es un sistema de interacción hombre-máquina basado en visión artificial diseñado para resolver el reto de la interpretación de gestos en tiempo real. El principal problema que aborda es la necesidad de procesar movimientos humanos de forma fluida y precisa bajo condiciones variables de luz y entorno.

La solución propuesta consiste en una aplicación que utiliza la cámara para detectar la mano del usuario, extraer sus puntos clave mediante MediaPipe y clasificar el gesto (Piedra, Papel o Tijera) usando un modelo de Machine Learning. El valor principal del proyecto es la creación de una experiencia de juego interactiva contra una IA competitiva, demostrando la integración práctica de técnicas de procesamiento de imagen, toma de decisiones automática y analítica de datos en un entorno ligero y funcional.

3. Descripción funcional del sistema

- <u>¿Qué hace exactamente la aplicación?</u>

  La aplicación es un sistema de visión artificial diseñado para la ejecución de una partida de "Piedra, Papel o Tijera" entre un usuario físico y un oponente virtual. El sistema captura el flujo de vídeo en tiempo real, extrae las características biométricas de la mano del jugador, clasifica el gesto realizado y ejecuta la lógica diseñada, gestionando el resultado del duelo y el marcador acumulado.
- <u>Flujo general de uso</u>
  - **Inicio**: El usuario ejecuta el programa/aplicación
  - **Validación:** El sistema verifica que el dispositivo está conectado y que las condiciones de luz son óptimas para la detección.
  - **Cuenta atrás:** El sistema muestra visualmente un contador (3, 2, 1...) para que el usuario prepare su gesto.
  - **Inferencia:** El sistema extrae los puntos de la mano y el modelo de IA identifica si es Piedra, Papel o Tijera.
  - **Resultado:** Se muestra la elección de la IA, el veredicto de la ronda y se actualiza el marcador global.
- <u>Casos principales de usuario</u>
  - **Jugador:** Usuario que busca competir contra la IA de forma rápida y sin mandos físicos.
- <u>Qué puede y qué no puede hacer el sistema</u>
  - **Puede:**
    1. Reconocer los tres gestos estándar con una sola mano en tiempo real.
    2. Funciona íntegramente en entornos locales.
    3. Adaptarse a diferentes morfologías de mano gracias al entrenamiento del modelo de IA.
    4. Gestionar un marcador acumulado y una lógica de juego competitiva de forma automática.
  - **No puede:**
    1. Detectar a dos jugadores físicos simultáneamente
    2. Funcionar correctamente si la mano está tapada, oculta o fuera del campo de visión
    3. Operar si las condiciones de luz son insuficientes o la cámara presenta fallos técnicos
    4. Interpretar gestos que no corresponden estrictamente a las tres categorías del juego

4. Alcance definitivo (IN / OUT)

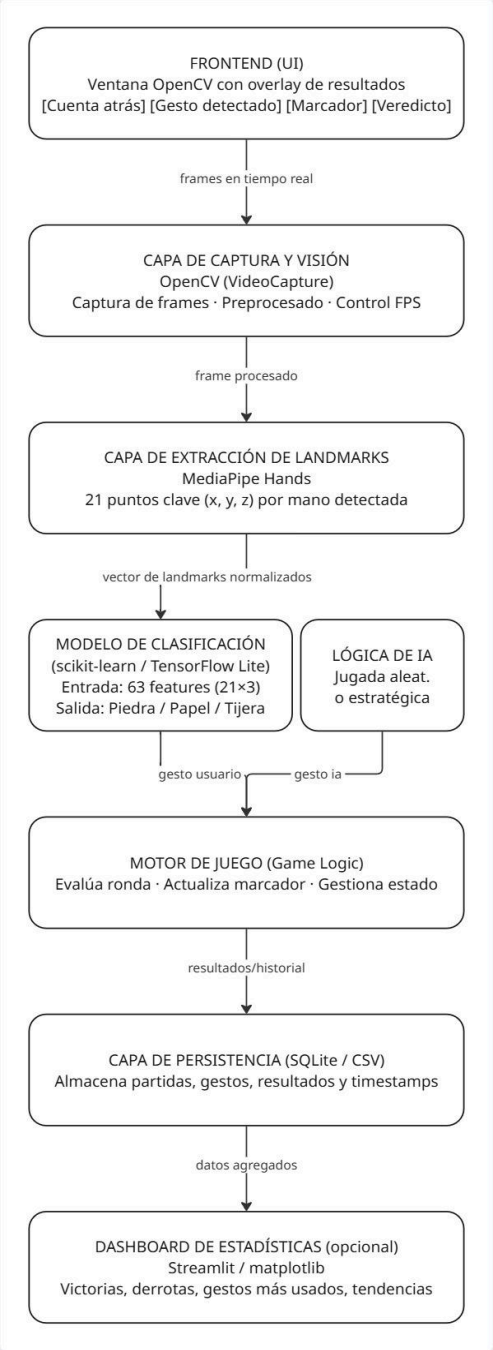
Incluye (IN)

- Funcionalidades mínimas:
    - Captura de vídeo y procesado de frames con OpenCV.
    - Uso de MediaPipe para la extracción de puntos clave de la mano.
    - Modelo entrenado para reconocer 3 clases: Piedra, Papel, Tijera.
    - Lógica de juego y sistema de puntuación.
    - Visualización de resultados mediante superposición en el vídeo
- Funcionalidades opcionales:
    - Dashboard sencillo para visualizar estadísticas de partidas

No incluye (OUT)

- Juego multijugador o en red.
- Reconocimiento de gestos de ambas manos simultáneamente.

5. Arquitectura del sistema (alto nivel)



- **Frontend (UI):** La interfaz se construye directamente sobre el feed de la cámara usando funciones de dibujo de OpenCV (cv2.putText, cv2.rectangle). Muestra la cuenta atrás, el gesto detectado del usuario, la jugada generada por la IA, el veredicto de la ronda y el marcador acumulado. No hay interfaz web; toda la interacción ocurre en la ventana de vídeo.
- **Capa de captura y visión:** OpenCV gestiona la conexión con la webcam, la captura de frames y el preprocesado básico (redimensionado, conversión de

espacio de color de BGR a RGB para MediaPipe). También controla el temporizador visual de cuenta atrás y determina el momento de inferencia.

- **Capa de extracción de landmarks:** MediaPipe Hands procesa cada frame y devuelve, cuando detecta una mano, un conjunto de 21 puntos tridimensionales (x, y, z) normalizados respecto al tamaño del frame. Este vector de 63 valores numéricos constituye la entrada del modelo de clasificación.
- **Modelo de clasificación (ML):** Recibe el vector de landmarks y lo clasifica en una de las tres categorías (Piedra, Papel, Tijera). Se entrenará con un dataset propio o público de landmarks etiquetados. Se contempla usar un clasificador ligero (RandomForest, SVM o red neuronal densa pequeña) que permita la inferencia en tiempo real sin GPU.
- **Lógica de IA (oponente):** Módulo que genera la jugada de la máquina. En su versión base, la elección es aleatoria (uniforme entre las tres opciones). Como mejora opcional, puede incorporar análisis de historial del usuario para predecir y contrarrestar sus patrones.
- **Motor de juego:** Contiene las reglas del juego (quién gana cada combinación), gestiona el estado de la partida (rondas jugadas, puntuación) y coordina el flujo entre los demás módulos.
- **Capa de persistencia:** Base de datos SQLite local (o ficheros CSV como alternativa mínima) donde se registran el timestamp, gesto del usuario, gesto de la IA y resultado de cada ronda. Permite recuperar el historial para el dashboard.
- **Dashboard de estadísticas (opcional):** Aplicación Streamlit o visualizaciones con matplotlib/seaborn que consume los datos almacenados y presenta métricas agregadas: ratio de victorias, distribución de gestos usados, evolución del marcador por sesión.

6. Tecnologías seleccionadas

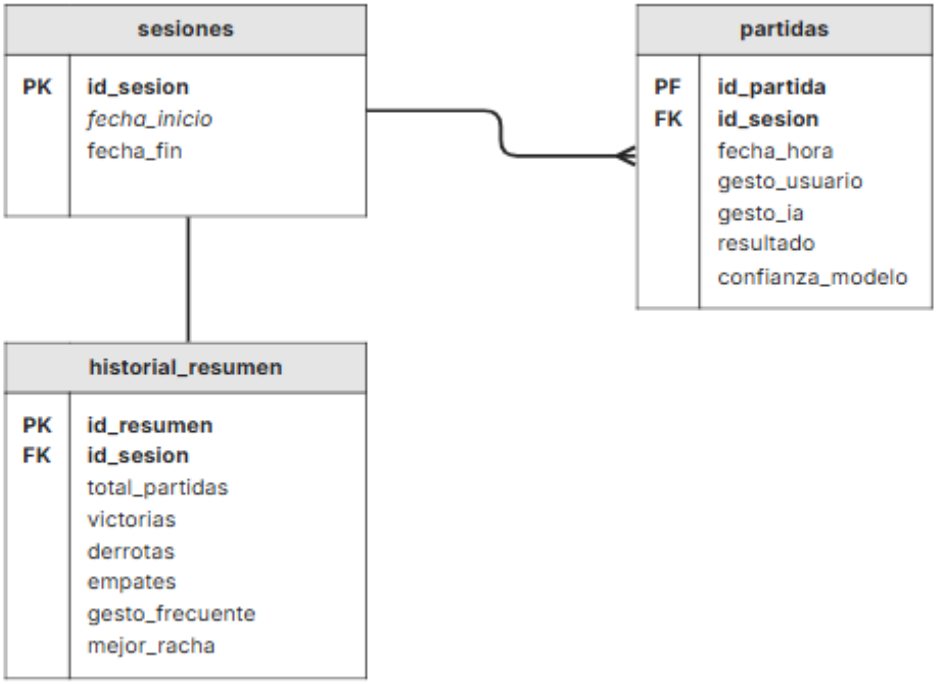
Componente	Tecnología elegida	Justificación	Alternativa descartada
Lenguaje principal	**Python 3.10+**	Ecosistema dominante en visión artificial y ML; amplia comunidad y documentación; compatibilidad con todas las librerías del stack	C++ (mayor rendimiento pero complejidad innecesaria para este alcance)
Captura y procesado de vídeo	**OpenCV 4.x**	Librería de referencia para visión artificial; soporta captura en tiempo real, manipulación de frames y rendering de overlays en una sola herramienta	Pygame (menos capacidades de visión)
Extracción de landmarks	**MediaPipe Hands (Google)**	Proporciona detección y tracking de 21 puntos de 3D de la mano, preentrenado y optimizado para CPU; elimina la necesidad de entrenar un detector propio	OpenPose (más pesado, requiere GPU para tiempo real)
Modelo de clasificación	**scikit-learn (RandomForest/SVM) o TensorFlow Lite**	scikit-learn permite prototipado rápido con modelos interceptables; TFLite como opción si se necesita precisión. Ambos operan en CPU sin dependencias pesadas.	PyTorch (overhead mayor modelos de esta escala)
Persistencia de datos	**SQLite (vía módulo sqlite3)**	Base de datos embebida, sin servidor, ideal para aplicaciones de escritorio de un solo usuario; no requiere infraestructura adicional	PostgreSQL (sobredimensionado para uso local)
Dashboard (opcional)	**Streamlit**	Permite construir dashboards interactivos en Python puto con mínimo código; se integra directamente con pandas y matplotlib	Dash/Plotly (mayor complejidad de configuración)
Visualización de datos	**matplotlib/seaborn**	Librerías estándar para gráficos estadísticos con Python; suficientes para las métricas previstas	Bokeh (innecesariamente interactivo para este caso)
Gestión de entorno	**pip + requirements.txt**	Solución ligera y universal; adecuada para un proyecto académico sin necesidad de gestión de paquetes compleja	conda (más adecuado para proyectos con dependencias científicas complejas)
Control de versiones	**Git + GitHub**	Estándar de la industria;	GitLab (funcionalidad equivalente

		facilita la colaboración entre integrantes del equipo y la revisión de código	pero menos familiar al equipo)

7. Modelo de datos

- **Diagrama entidad-relación**

  El sistema almacenará, tanto los datos generados durante las partidas, como cierta información agregada para su posterior análisis.



- **Explicación de las tablas**

Tabla: sesiones		
Campo	**Tipo**	**Descripción**
id_sesion	INTEGER (PK)	ID único de la sesión
fecha_inicio	DATETIME	Momento de inicio de la sesión
fecha_fin	DATETIME	Momento de finalización de la sesión

Tabla: partidas		
Campo	**Tipo**	**Descripción**
id_partida	INTEGER (PK)	ID único de la partida
id_sesion	INTEGER (FK)	ID en relación con la sesión
fecha_hora	DATETIME	Momento en el que se juega la ronda
gesto_usuario	TEXT	Gesto detectado (Piedra, Papel o Tijera)
gesto_ia	TEXT	Gesto generado por la IA
resultado	TEXT	Resultado de la ronda (Victoria, Derrota, Empate)
confianza_modelo	FLOAT	Nivel de confianza de la predicción del modelo

Tabla: historial_resumen		
Campo	**Tipo**	**Descripción**
id_resumen	INTEGER (PK)	ID único del historial/resumen
id_sesion	INTEGER (FK)	ID en relación con la sesión
total_partidas	INTEGER	Número total de partidas jugadas
victorias	INTEGER	Número de victorias del usuario
derrotas	INTEGER	Número de derrotas del usuario
empates	INTEGER	Número de empates del usuario
gesto_frecuente	TEXT	Gesto más frecuente en la sesión
mejor_racha	INTEGER	Máxima racha de victorias consecutivas

- **Relaciones**
  - Relación 1:N entre **sesiones** y **partidas**.
  - Relación 1:1 entre **sesiones** y **historial_resumen**.

  Vamos a separar, por un lado los datos transaccionales (partidas) y los datos agregados para posibles estudios estadísticos (historial).

- **Claves**

Tabla	Clave primaria (PK)	Clave foránea (FK)
sesiones	id_sesion	-
partidas	id_partida	id_sesion
historial_resumen	id_resumen	id_sesion

- **Dataset (diccionario de variables)**
  Nuestro modelo utilizará un dataset estructurado basado en landmarks de la mano, procedente, tanto de fuente externas, como de datos generados por el equipo.

Cada registro contiene:

Variable	Tipo	Descripción
x1, y1, z1 ... x21, y21, z21	FLOAT	Coordenadas tridimensionales de los 21 puntos clave de la mano
gesto	TEXT	Clase del gesto (Piedra, Papel o Tijera)

En total habrá 63 variables numéricas (21 puntos x 3 coordenadas) y 1 variable objetivo (gesto).
Este enfoque nos permite trabajar con datos estructurados (gracias a MediaPipe), en lugar de imágenes, reduciendo la complejidad del modelo y facilitando la ejecución en tiempo real.

8. Datos

- **Fuentes**
  El sistema utilizará como base un dataset público disponible en Kaggle, concretamente el Stone-Paper-Scissors Hand Landmarks Dataset, que contiene coordenadas de puntos clave de la mano (landmarks) etiquetadas según el gesto realizado.

  Este dataset es compatible con el pipeline del sistema, ya que proporciona directamente las características necesarias para el modelo de clasificación.

Además, se contempla la generación de un dataset propio mediante captura de landmarks con MediaPipe, con el objetivo de ampliar y mejorar la calidad de los datos.

- **Licencia**
  El dataset utilizado es de acceso público, a través de Kaggle. En caso de necesitar ampliar los datos, se hará con datos propios, generados internamente por el equipo y sin restricciones de uso.

- **Tipo de datos**
  El sistema trabaja con datos estructurados, basados en landmarks de la mano. Se distinguen 3 tipos:
  - Datos de entrada: coordenadas tridimensionales (x,y,z) de los 21 puntos clave de la mano.
  - Datos etiquetados: clase asociada al gesto (piedra, papel o tijera)
  - Datos derivados: resultados de clasificación generados por el modelo.

- **Volumen estimado**
  El dataset base de Kaggle contiene un volumen significativo de muestras por cada una de las clases (Piedra, Papel y Tijera), pudiendo alcanzar decenas de miles de registros en función de la versión utilizada. En caso de necesidad de ampliación, el volumen lo podremos incrementar progresivamente los miembros del propio grupo.

- **Frecuencia de actualización**
  - El dataset base será estático, por lo que no debería tener actualizaciones.
  - La base de datos del sistema se actualizará en tiempo real durante cada partida.

- **Calidad esperada**
  La calidad de los datos suponemos que es adecuada para un modelo de clasificación supervisado como este, aunque podrían existir diferentes factores que pueden afectar al rendimiento, como la variabilidad entre usuarios, diferencias en la ejecución de los gestos o posibles errores con la detección de los landmarks.

- **Riesgo de datos**

Sesgos en los datos		
Riesgo	**Impacto**	**Mitigación**
Distribución desigual de clases o predominancia de un tipo de mano.	El modelo puede favorecer ciertas clases o fallar en generalización.	Balanceo de clases y uso de múltiples usuarios en la generación de datos.
Ruido en los datos		
Riesgo	**Impacto**	**Mitigación**
Errores en la detección de landmarks o posiciones ambiguas de la mano.	Datos inconsistentes que afectan al entrenamiento.	Filtrado de muestras incorrectas y validación del dataset.
Dataset insuficiente o poco representativo		
Riesgo	**Impacto**	**Mitigación**
El dataset puede no cubrir suficiente variabilidad de usuarios o condiciones.	Reducción en la precisión del modelo.	Ampliación del dataset mediante captura de datos propios.

9. Ética, legalidad y accesibilidad

- **Presencia o ausencia de datos personales (PII)**: Durante la ejecución del juego, la capa de captura visual de OpenCV procesa vídeo en tiempo real que incluye rostros, características biométricas de la mano y el entorno del usuario, lo que constituye información personal identificable (PII). Sin embargo, la capa de extracción utiliza MediaPipe para transformar la imagen de la mano en un vector de 63 características numéricas (21 puntos clave en 3D). En cuanto a la persistencia, la base de datos SQLite está diseñada para almacenar únicamente datos anónimos de la partida (timestamps, gesto del usuario, gesto de la IA y resultados), sin guardar en ningún momento fotogramas ni vídeo del usuario. Solo existirá almacenamiento de PII en la fase previa de creación del dataset propio para el entrenamiento del modelo.

- **Cumplimiento GDPR**: En el entorno de producción (durante el juego), el sistema es altamente respetuoso con la privacidad, ya que procesa el feed de vídeo de forma efímera (en memoria) para extraer los landmarks y no almacena las imágenes en disco. Para cumplir con el Reglamento General de Protección de Datos (GDPR) durante la fase de recolección de datos, se deberá recabar el consentimiento explícito de las personas (entre 3 y 5 individuos distintos) cuyas manos sean grabadas para la construcción del dataset de entrenamiento.

- **Riesgos de sesgo**: Tal y como se ha identificado en el análisis de riesgos técnicos (R4), existe un riesgo alto de que el modelo carezca de generalización si el dataset de entrenamiento es insuficiente, está desbalanceado o contiene capturas de un solo tipo de mano o de una única persona. Asimismo, las variaciones morfológicas entre distintos usuarios pueden provocar clasificaciones incorrectas (riesgo R2), especialmente entre gestos similares como la "Piedra" y un puño relajado.

- **Medidas de mitigación**: Para mitigar los sesgos en la clasificación (R4), el plan de acción consiste en recopilar muestras de al menos 3 a 5 personas diferentes, asegurando el balanceo de las clases (Piedra, Papel, Tijera) y documentando las condiciones de iluminación de las capturas. Adicionalmente (R2), se aplicará una validación en la lógica del sistema incorporando un umbral de confianza mínimo (ej. >80%); si la predicción del modelo no supera este margen de seguridad ante las variaciones del usuario, el sistema no forzará un resultado y mostrará el mensaje "gesto no reconocido".

- **Accesibilidad básica prevista**: La interfaz del juego (Frontend) no requiere de navegación web ni periféricos tradicionales durante la partida. Todo se desarrolla sobre una ventana de vídeo de OpenCV interactuando exclusivamente mediante gestos corporales de una mano. Para garantizar la accesibilidad visual, el sistema utilizará funciones de renderizado gráfico (cv2.putText, cv2.rectangle) que superpondrán información clara y de gran tamaño en pantalla, incluyendo una cuenta atrás visual, el gesto detectado, el marcador acumulado y el veredicto explícito de cada ronda.

10. Backlog inicial

- **US 01 | Captura de vídeo en tiempo real (Platform / Data)**
  - <u>Descripción:</u>
    Como sistema, necesito conectarme a la cámara web y capturar el

flujo de vídeo en tiempo real para poder procesar los frames mediante visión artificial.

- Criterios de aceptación:
  El script en Python inicializa la cámara usando OpenCV (VideoCapture) sin errores, mantiene una tasa de frames fluida y preprocesa la imagen (por ejemplo, conversión de color BGR a RGB) de forma correcta.

- **US 02 | Extracción de puntos clave de la mano (Data)**
  - Descripción:
    Como sistema, quiero utilizar MediaPipe Hands sobre los frames capturados para extraer las coordenadas de la mano y usarlas como entrada para el modelo.
  - Criterios de aceptación:
    Cuando una mano aparece en cámara, MediaPipe detecta y extrae correctamente un vector de 21 puntos clave tridimensionales (63 características numéricas en total) y los dibuja sobre el frame para su confirmación visual.

- **US 03 | Creación del dataset de entrenamiento (Data)**
  - Descripción:
    Como Data Engineer, necesito capturar muestras de gestos (Piedra, Papel, Tijera) de distintos usuarios para tener datos representativos con los que entrenar el modelo predictivo.
  - Criterios de aceptación:
    Se genera un archivo (ej. CSV) que contiene vectores de landmarks etiquetados con su clase correspondiente, incluyendo muestras de al menos 3 a 5 personas diferentes para garantizar la generalización y evitar sesgos.

- **US 04 | Entrenamiento del modelo de clasificación (ML)**
  - Descripción:
    Como ML Engineer, quiero entrenar un modelo ligero (ej. RandomForest, SVM o TensorFlow Lite) usando el dataset de landmarks para que el sistema aprenda a clasificar los tres gestos posibles.
  - Criterios de aceptación:
    El modelo se entrena exitosamente, puede ejecutarse en CPU y demuestra una alta precisión en las métricas de validación (matriz de confusión), superando el umbral mínimo de confianza del 80%.

- **US 05 | Lógica del oponente virtual (Platform)**
  - Descripción:
    Como jugador, quiero que la IA genere una jugada propia (Piedra, Papel o Tijera) en cada ronda para poder competir contra ella.
  - Criterios de aceptación:
    El motor de juego incluye una función que devuelve una elección válida y aleatoria para la máquina una vez que finaliza la cuenta atrás de la ronda.

- **US 06 | Motor de juego y resolución de la ronda (Platform)**
  - Descripción:
    Como jugador, quiero que el sistema compare mi gesto detectado con el generado por la IA para determinar justamente quién ha ganado la ronda.
  - Criterios de aceptación:
    El motor evalúa correctamente la lógica clásica de Piedra-Papel-Tijera en sus 9 combinaciones posibles, determinando si el estado de la ronda es Victoria, Derrota o Empate, y actualizando el marcador global.

- **US 07 | Interfaz gráfica y superposición visual (Platform)**
  - Descripción:
    Como jugador, necesito ver en pantalla un temporizador y el resultado de la partida superpuesto en el vídeo para saber cuándo hacer mi gesto y cuál ha sido el veredicto.
  - Criterios de aceptación:
    Usando funciones de OpenCV (cv2.putText, cv2.rectangle), la pantalla muestra de forma legible la cuenta atrás (3, 2, 1), el gesto reconocido, la jugada de la IA, el veredicto y el marcador acumulado.

- **US 08 | Gestión de gestos no reconocidos (ML / Platform)**
  - Descripción: Como jugador, quiero que el sistema me avise si no ha entendido mi gesto o si la mano no se ve bien, en lugar de asignarse una jugada al azar que me haga perder.
  - Criterios de aceptación: Si la probabilidad de predicción del modelo es inferior al umbral del 80%, o si MediaPipe no detecta una mano en el frame clave, la interfaz muestra el mensaje "gesto no reconocido" y la ronda se anula sin afectar al marcador.

- **US 09 | Persistencia del historial de partidas (Platform / BI)**
    - Descripción: Como analista, necesito que los resultados de cada ronda se guarden de forma persistente localmente para su posterior análisis estadístico.
    - Criterios de aceptación: Al finalizar cada ronda, se inserta un nuevo registro en una base de datos SQLite (o archivo CSV alternativo) que contiene el timestamp, el gesto del usuario, la elección de la IA y el resultado.

- **US 10 | Dashboard de analítica (BI - Opcional)**
    - Descripción:
      Como usuario, quiero acceder a un panel interactivo para visualizar las estadísticas históricas, tendencias y porcentaje de victorias frente a la máquina.
    - Criterios de aceptación:
      Se crea una aplicación con Streamlit que lee los datos de SQLite y renderiza al menos dos gráficas descriptivas (ej. ratio de victorias con matplotlib/seaborn y distribución de gestos utilizados).

11. Plan de trabajo

Reparto de tareas

El equipo ha asignado los roles basándose en la arquitectura del sistema:

- **Javier Quiñonero** (Data Engineer): Responsable de la capa de captura y visión. Se encargará de la captura del dataset inicial y de implementar la extracción de coordenadas (landmarks) utilizando MediaPipe.
- **David Bartol** (ML Engineer): Responsable del modelo de clasificación. Entrenará el algoritmo (scikit-learn o TensorFlow Lite) con los landmarks extraídos y validará las métricas de rendimiento (matriz de confusión).
- **Daniel Chacón** (Platform Architect): Responsable del motor de juego y el frontend. Integrará la cámara, la IA y la lógica del juego (Piedra, Papel o Tijera), además de diseñar la interfaz gráfica superpuesta mediante OpenCV.
- **Álvaro Benito** (BI & QA Analyst): Responsable de la persistencia y visualización de datos. Implementará el registro de las partidas en SQLite y diseñará el dashboard de rendimiento con Streamlit.

Dependencias

El flujo de trabajo es secuencial en las primeras fases, lo que genera dependencias claras entre los roles:

- Dependencia de Datos-ML: El trabajo de entrenamiento del modelo (David) está bloqueado hasta que el sistema de extracción de MediaPipe y el dataset inicial (Javier) estén operativos.
- Dependencia ML-Platform: El motor de juego final y la interfaz (Daniel) necesitan el modelo de clasificación entrenado (David) para poder ejecutar la inferencia en tiempo real y resolver las rondas.
- Dependencia Platform-BI: El dashboard de analíticas (Álvaro) requiere que el motor de juego (Daniel) esté generando y guardando correctamente los resultados en la base de datos SQLite.

Cronograma por semanas e hitos principales

Planificación estimada para 4 sprints.

- Semana 1 | Captura y Datos:
  Configuración del entorno (requirements.txt), pruebas de captura de vídeo con OpenCV y extracción de puntos con MediaPipe. Recolección de muestras para el dataset.
  Hito: Dataset consolidado y pipeline de visión artificial operativo.

- Semana 2 | Modelado ML:
  Entrenamiento del modelo de clasificación, ajuste de hiperparámetros y evaluación para superar el umbral mínimo de confianza del 80%.
  Hito: Modelo ML validado y empaquetado para su uso local.
- Semana 3 | Integración y Lógica:
  Desarrollo del motor de juego, implementación de la jugada automática de la IA y superposición de la interfaz (cuenta atrás y resultados) sobre el vídeo en tiempo real.
  Hito: MVP jugable (Producto Mínimo Viable) completado.
- Semana 4 | Persistencia, Analítica y QA:
  Conexión de la base de datos SQLite, desarrollo del dashboard en Streamlit, y testeo funcional cruzado entre todos los integrantes.
  Hito: Release final del proyecto EmoGestureAI y despliegue del Dashboard.

Resumen de Riesgos Técnicos

Se tendrán en cuenta los siguientes riesgos críticos durante el desarrollo:

- **R1 & R8:** Problemas de hardware (iluminación deficiente o acceso bloqueado a la cámara web).
- **R2 & R4:** Fallos del modelo por similitud de gestos o por un dataset sesgado y poco diverso.
- **R3:** Problemas de rendimiento y latencia elevada al usar procesamiento por CPU.

- R5: Incompatibilidades en las versiones de librerías entre los equipos de los desarrolladores.
- R6: Corrupción de datos en el guardado de partidas en SQLite.
- R7: Alcance mal dimensionado que retrase el núcleo del proyecto por dar prioridad a funciones opcionales (Dashboard).

12. Estrategia de desarrollo

Para garantizar la calidad del software y la estabilidad del proyecto, el equipo seguirá una metodología basada en revisiones conjuntas y pruebas antes de cada integración.

- Flujo Git (branches)
  Se utilizará una adaptación del modelo **GitFlow**. Dónde queda prohibido realizar push directos sobre la rama main.
  - **Rama main:** Contiene la versión estable y funcional del juego
  - **Rama develop:** Rama de integración donde se unen las funcionalidades terminadas.
  - **Ramas feature/:** Cada nueva funcionalidad tendrá su propia rama independiente.
  - **Pull Requests (PR)**: Al finalizar el desarrollo, se abrirá una PR hacia develop. Esta requerirá obligatoriamente la **aprobación unánime** de todos los integrantes del equipo tras una revisión de código.
- Integración del código
  - **Revisión técnica:** Se comprobará que el nuevo código sigue los estándares de nombrado y documentación (Docstrings)
  - **Resolución de conflictos:** El responsable de la rama deberá asegurar que su código está actualizado con la última versión de develop antes de fusionar.
- Validación de funcionalidad
  - Todos los integrantes probarán la nueva funcionalidad en sus propios entornos locales.
  - Se buscarán activamente errores de lógica o fallos de ejecución que puedan comprometer la versión.
- Testing básico
  - **Test Unitarios:** Creación de pequeños scripts de prueba para verificar funciones aisladas devuelven los valores esperados.
  - **Pruebas de Historias de Usuario (US):** Para cada US definida en el backlog, se ejecutará una prueba funcional que valide que se cumplen todos los “Criterios de Aceptación” antes de dar la tarea por finalizada.

13. Riesgos técnicos actualizados

#	Riesgo	Impacto	Probabilidad	Plan de mitigación
R1	Detección deficiente de la mano en condiciones de luz inadecuadas o fondos complejos	Alto - el sistema no puede funcionar sin landmark detection	Media	Añadir validación de condiciones al inicio (brightness check); incluir guía visual al usuario; probar con iluminación artificial controlada en demos
R2	Clasificación incorrecta del gesto por similitud entre Piedra y Puño relajado, o variaciones entre usuarios	Alto - produce resultados de juego incorrectos y deteriora la experiencia	Media - Alta	Ampliar el dataset de entrenamiento con muestras de distintos usuarios; incorporar umbral de confianza mínimo (ej. >80%) y mostrar "gesto no reconocido" si no se supera
R3	Latencia elevada que impide la inferencia en tiempo real en hardware con CPU limitada	Medio - puede hacer la aplicación injugable	Baja - Media	Limitar la resolución del frame de entrada (ej. 640x480); procesar la inferencia solo en el frame del momento de cuenta atrás; perfilar el pipeline desde el inicio
R4	Dataset insuficiente o sesgado (pocas muestras, un solo tipo de mano, una sola persona) que provoca baja generalización del modelo	Alto - el modelo no funcionará con usuarios distintos al que se entrenó	Media	Recopilar muestras del al menos 3-5 personas distintas; balancear las clases; documentar las condiciones de captura
R5	Dependencia de versiones específicas de MediaPipe u OpenCV que generan incompatibilidades entre los entornos de los distintos integrantes	Medio - bloquea el desarrollo en algún equipo	Baja	Fijar versiones exactas en requirements.txt desde el inicio; documentar la versión de Python usada; considerar uso de entorno virtual compartido
R6	Pérdida o corrupción de datos de partidas almacenadas en SQLite por errores de escritura o cierre abrupto	Bajo - solo afecta al historial, no a la funcionalidad principal	Baja	Usar transacciones SQLite correctamente; añadir manejo de excepciones en las operaciones de escritura; hacer commit periódico
R7	Alcance mal dimensionado: la	Medio - puede dejar el proyecto	Media	Tratar todas las funcionalidades

	integración del módulo de predicción de patrones o el dashboard consume más tiempo del estimado y compromete las funcionalidades core	incompleto		opcionales como mejoras desacopladas del core; no iniciar ninguna hasta que el flujo principal esté validado y testeado
R8	Problemas de acceso a cámara en algunos sistemas operativos (permisos en macOS/Windows, drivers en Linux)	Medio - impide ejecutar la aplicación en el entorno de demo	Baja	Documentar el proceso de configuración de permisos; probar en los SO de todos los integrantes en la primera semana; preparar vídeo de demostración grabado como fallback

14. Repositorio del proyecto

https://github.com/danielchb01/GesturaAI